

Федеральное государственное автономное образовательное учреждение
высшего образования

**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»**

Отчет
по лабораторной работе №2.4
«Алгоритмы на строках»
по дисциплине «Алгоритмы и структуры данных»

Автор: Бен Шамех Абделаиз

Факультет: ПИН

Группа: K3239

Преподаватель: Ромакина Оксана Михайловна

Санкт-Петербург 2026

1. Цель работы

Цель лабораторной работы — освоение алгоритмов работы со строками: наивный поиск подстроки, подсчёт палиндромных троек с DP, Z-функция, а также нахождение наибольшей общей подстроки.

2. Задание на лабораторную работу (Вариант 27)

- Задача 2. Карта — подсчитать число трёхбуквенных палиндромов (подслов АВА). [1 балл]
- Задача 6. Z-функция — построить Z-функцию для заданной строки. [1.5 балла]
- Задача 7. Наибольшая общая подстрока — найти LCS двух строк (DP). [2 балла]
- Задача 1*. Наивный поиск подстроки — найти все вхождения Р в Т (доп.). [1 балл]

3. Краткие теоретические сведения

Z-функция $z[i]$ строки s — длина наибольшего префикса s , совпадающего с подстрокой $s[i..]$. Вычисляется за $O(|s|)$ через Z-блок $[l, r]$. $z[0] = |s|$ по определению.

Наибольшая общая подстрока (LCS): $dp[i][j]$ — длина наибольшей общей подстроки, оканчивающейся на $p[i-1]$ и $t[j-1]$. Если $p[i-1] == t[j-1]$: $dp[i][j] = dp[i-1][j-1] + 1$, иначе 0. Ответ — максимум таблицы. Сложность $O(n*m)$.

Задача «Карта»: число способов выбрать три позиции $i < j < k$ такие, что $s[i] == s[k]$. Для каждой средней позиции j считаем произведение числа совпадающих символов слева и справа.

4. Выполнение работы

4.1. Задача 2 — Карта (трёхбуквенные палиндромы)

DP: $dp[i][j]$ — длина наибольшей общей подпоследовательности $s[i..j]$. Если $s[i] == s[j]$: $dp[i][j] = dp[i+1][j-1] + 1$, иначе $\max(dp[i+1][j], dp[i][j-1])$. Ответ —

`dp[0][n-1]`.

Листинг 1 — task2.py

```
def count_ways_to_form_palindrome(s):
    n = len(s)
    dp = [[0]*n for _ in range(n)]
    for i in range(n):
        dp[i][i] = 1
    for length in range(2, n+1):
        for i in range(n-length+1):
            j = i+length-1
            if s[i] == s[j]:
                dp[i][j] = dp[i+1][j-1] + 1
            else:
                dp[i][j] = max(dp[i+1][j], dp[i][j-1])
    return dp[0][n-1]
```

●●● Рисунок 1 — Задача 2: Карта (трёхбуквенные палиндромы)

```
$ python3 task2.py
Тест 1: s="treasure" -> dp[0][7]=2    output.txt: 2
Тест 2: s="you will never find the treasure"
        output.txt: 146
```

Рисунок 1 — Демонстрация работы: Задача 2 (Карта)

4.2. Задача 6 — Z-функция

Z-функция $z[i]$ = длина наибольшего совпадения $s[0..]$ с $s[i..]$. Алгоритм поддерживает Z-блок $[l, r]$. При $i \leq r$ используем $z[i-l]$, иначе вычисляем наивно и обновляем $[l, r]$. $z[0] = n$. Сложность $O(n)$.

Листинг 2 — task6.py

```
def z_function(s):
    n = len(s)
    if n == 0: return []
    z = [0]*n
    z[0] = n
    l, r = 0, 0
    for i in range(1, n):
        if i <= r:
            k = i - l
            if s[k] == s[i]:
                z[i] = z[k]
            else:
                l, r = i, i
                while r < n and s[r] == s[r-l]: r += 1
                z[i] = r - l; r -= 1
        else:
            l, r = i, i
            while r < n and s[r] == s[r-l]: r += 1
            z[i] = r - l; r -= 1
    return z
```

●●● Рисунок 2 — Задача 6: Z-функция

```
$ python3 task6.py
Тест 1: s="aaaAAA"
z = [6, 2, 1, 0, 0, 0]
output.txt: 6 2 1 0 0 0
Тест 2: s="abacaba"
z = [7, 0, 1, 0, 3, 0, 1]
output.txt: 7 0 1 0 3 0 1
```

Рисунок 2 — Демонстрация работы: Задача 6 (Z-функция)

4.3. Задача 7 — Наибольшая общая подстрока

DP: $dp[i][j]$ — длина наибольшей общей подстроки, оканчивающейся на $p[i-1]$ и $t[j-1]$. Если символы совпадают: $dp[i][j] = dp[i-1][j-1] + 1$, иначе 0. Выводим: $start_pos_p$, $start_pos_t$, max_len .

Листинг 3 — task7.py

```
def longest_common_substring(p, t):
    n, m = len(p), len(t)
    dp = [[0]*(m+1) for _ in range(n+1)]
    max_len = 0; end_pos_p = 0
    for i in range(1, n+1):
        for j in range(1, m+1):
            if p[i-1] == t[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
                if dp[i][j] > max_len:
                    max_len = dp[i][j]
                    end_pos_p = i
            else:
                dp[i][j] = 0
    start_pos_p = end_pos_p - max_len
    start_pos_t = t.find(p[start_pos_p:start_pos_p+max_len])
    return start_pos_p, start_pos_t, max_len
```

●●● Рисунок 3 — Задача 7: Наибольшая общая подстрока

```
$ python3 task7.py
Тест 1: p="cool", t="toolbox"
max_len=3 ("ool"), start_pos_p=1, start_pos_t=1
output.txt: 1 1 3
Тест 2: p="aaa", t="bb" -> max_len=0
output.txt: 0 0 0
```

Рисунок 3 — Демонстрация работы: Задача 7 (Наибольшая общая подстрока)

4.4*. Задача 1 — Наивный поиск подстроки (дополнительное задание)

Наивный алгоритм: для каждой позиции i в тексте T проверяем совпадение $T[i:i+|P|]$ с P . При совпадении добавляем $i+1$ (нумерация с 1) в список. Выводим: количество вхождений, затем позиции через пробел. Сложность $O(|T|*|P|)$.

Листинг 4 — task1_naive.py

```
def naive_substring_search(p, t):
    n, m = len(t), len(p)
    matches = []
    for i in range(n-m+1):
        if t[i:i+m] == p:
            matches.append(i+1)
    return len(matches), matches
p, t = read_input_file(file_path1)
count, matches = naive_substring_search(p, t)
result = f"{count}\n"
if matches:
    result += " ".join(map(str, matches))
```

●●● Рисунок 4 — Задача 1: Наивный поиск подстроки

```
$ python3 task1_naive.py
p="aba", t="abaCaba"
i=0: "aba"==p -> pos 1
i=4: "aba"==p -> pos 5
output.txt:
2
1 5
```

Рисунок 4 — Демонстрация работы: Задача 1 (Наивный поиск подстроки)

5. Вывод

В ходе выполнения лабораторной работы изучены и реализованы алгоритмы на строках. Z-функция позволяет за $O(n)$ получить информацию о всех совпадениях подстрок с префиксом. DP за $O(n*m)$ находит наибольшую общую подстроку двух строк с точными позициями. Задача «Карта» решена через DP на подпоследовательностях. Наивный поиск за $O(n*m)$ корректен для малых входных данных.